聚类分析（Cluster Analysis）

金融大数据

课程目标

1. **理解聚类的基本概念**：什么是聚类、为什么需要聚类
2. **掌握相似性度量**：距离公式、标准化方法
3. **学习核心算法**：K-Means、层次聚类、DBSCAN
4. **能够选择合适算法**：根据数据结构选择聚类方法

课程大纲

第一部分：基本概念

- 什么是聚类分析
- 聚类 vs 分类
- 簇内相似度与簇间相似度
- 距离度量与数据标准化
- 簇的形状与类型

第二部分：原理与算法

- K-Means 聚类算法
- 层次聚类
- DBSCAN 密度聚类
- 算法对比与选择指南

第一部分：基本概念

1.1 什么是聚类分析

聚类分析的定义

聚类分析（Cluster Analysis）是一种无监督学习方法，其目标是：

> 将一组数据对象划分为若干个子集（称为"**簇**"），使得**同一簇内**的对象彼此**相似**，而**不同簇之间**的对象**相异**。

核心思想："物以类聚，人以群分"

通俗理解：水果分类

想象你有一堆水果，但你不知道它们的种类：

- 观察特征：颜色、大小、形状、重量
- 自然分组：苹果一堆、香蕉一堆、橙子一堆

聚类就是让数据"自己说话"，自动发现内在结构！

聚类 vs 分类

维度	聚类（Clustering）	分类（Classification）
学习类型	无监督学习	有监督学习
训练数据	无标签数据	有标签数据
目标	发现数据结构	预测新数据类别
先验知识	不需要知道类别数	需要预先知道类别

关键区别：

- 分类："教机器认识已知类别" → 告诉机器什么是苹果、什么是香蕉
- 聚类："让机器自己发现类别" → 只给一堆水果，让机器自己找出分组规律

聚类的核心目标

最大化簇内相似度，最小化簇间相似度

- **簇内相似度高**：同一簇内的数据点彼此距离较近
- **簇间相似度低**：不同簇的数据点彼此距离较远

好的聚类 = 组内"抱团紧密"，组间"分离清楚"

生活化例子：班级分组

好的分组结果：

- 小组 A：全是 90 分以上的学霸
- 小组 B：成绩 80-89 分的优秀学生
- 小组 C：成绩 70-79 分的中等学生
- 小组 D：成绩 60-69 分的及格边缘学生

簇内相似度高（组内成绩接近）+ 簇间相似度低（组间差距明显）✅

差的分组示例

差的分组结果：

- 小组 A：95分、72分、88分、61分（成绩跨度大）
- 小组 B：93分、70分、85分、63分（同样跨度大）

簇内相似度低（组内参差不齐）+ 簇间相似度高 ✗（组间难以区分）

可视化：好聚类 vs 差聚类

```
好的聚类：                          差的聚类：
  ●●●                                 ●    ●
  ●●●     ▲▲▲▲                      ●    ▲   ▲
  ●●       ▲▲▲                    ●   ▲▲    ●   ▲
           ▲▲                         ▲     ▲
```

- 左图：两个簇各自抱团，彼此分离 ✓
- 右图：点分布散乱，两个簇混在一起 ✗

聚类质量评价

数学直觉：

$$\text{聚类质量} \propto \frac{\text{簇间距离}}{\text{簇内距离}}$$

这个比值越大，说明聚类效果越好——簇间差异远大于簇内差异。

类比方差分析（ANOVA）：

- 组内方差小 → 组内数据一致
- 组间方差大 → 组间差异显著
- F统计量 = 组间方差 / 组内方差 → 值越大，分组越有意义

聚类的典型应用场景

1. **市场营销**：客户细分（价格敏感型、品质追求型）
2. **图像识别**：图像分割（像素按颜色、纹理聚类）
3. **生物信息学**：基因表达数据分析
4. **社交网络**：社区结构发现
5. **异常检测**：信用卡欺诈检测

1.5 相似性的度量：距离

聚类分析的核心是"相似度"的量化，最常用的方式是通过**距离**来衡量。

距离越小，相似度越高。

为什么需要多种距离度量？

核心问题：不同的距离度量关注数据的不同方面。

例子：评价两个学生是否"相似"

- 学生A：数学100分，语文100分
- 学生B：数学60分，语文60分
- 学生C：数学100分，语文60分

关注点	结果	距离类型
绝对水平	A和B差异大（总分差80分）	欧氏距离
学科倾向	A和C都是"偏数学"类型	余弦相似度

选择不同的距离，会得到完全不同的聚类结果！

欧氏距离（Euclidean Distance）

两点之间的直线长度，就是你拿尺子量的距离。

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

生活化例子：商场顾客口味相似度

- 你：每周买3次水果，花200元
- 顾客A：每周买4次水果，花250元
- 你和A的距离 ≈ 50
- 你和B（买1次，花50元）的距离 ≈ 150

→ 你和A更相似，系统会把你们聚为一类

曼哈顿距离（Manhattan Distance）

想象在网格状城市中从一个路口到另一个路口的距离（只能沿街走）。

$$d(x, y) = \sum_{i=1}^{n} |x_i - y_i|$$

为什么叫"曼哈顿"？

从A到B：欧氏距离是直线，曼哈顿距离是沿街走的步数之和。

区别：欧氏距离对大差异更敏感（因为要平方），曼哈顿距离对异常值更稳健。

余弦相似度（Cosine Similarity）

核心理解：关注"模式相似"，不关注"大小差异"。

$$\text{cosine}(x, y) = \frac{x \cdot y}{\|x\| \cdot \|y\|}$$

例子：文本相似度比较

文章	AI	机器学习	深度学习
A（AI主题）	100次	50次	30次
B（AI主题，更长）	200次	100次	60次
C（美食主题）	5次	3次	好吃80次

- 欧氏距离：A和B距离很大（词频差2倍）→ ❌

  但它们是同一主题！
- 余弦相似度：A和B ≈ 1.0（词频比例相同）→ ✅

余弦相似度：箭头类比

把每篇文章看作一个箭头（向量）：

- 箭头的**方向** = 文章的"主题倾向"
- 箭头的**长度** = 文章的"篇幅大小"

余弦相似度只看"箭头指向是否相同"，不管"箭头有多长"。

适用场景：

- 文本挖掘（文档相似度比较）
- 推荐系统（用户偏好相似度）
- 当关注方向/模式而非大小时

1.6 数据标准化

为什么需要标准化?

因为不同的特征可能有不同的量纲和取值范围。如果不做标准化，数值范围大的特征会在距离计算中占据主导地位。

极端例子：年龄 vs 年收入

客户	年龄	年收入（元）
A	25	8,000
B	30	12,000
C	45	500,000

- 年龄差异（5岁、15岁）被收入差异（几千元 vs 几十万元）**完全淹没**!
- "收入差异"贡献了99.999%的距离，"年龄差异"只占0.001%

标准化：让所有人站在同一起跑线

标准化之后，每个特征都变成"均值为0，标准差为1"：

- 年龄：25岁、30岁、45岁 → -1.0, -0.5, +1.5（标准分）
- 收入：8000、12000、500000 → -0.7, -0.7, +1.4（标准分）

现在算距离，年龄和收入各贡献一半，两个特征都有发言权。

常用方法：

- **Z-Score 标准化**：$x' = \frac{x-\mu}{\sigma}$ → 均值0，标准差1
- **Min-Max 归一化**：$x' = \frac{x-\min(x)}{\max(x)-\min(x)}$ → 落在[0, 1]区间

1.7 簇的形状与类型

为什么簇的形状很重要?

不同的聚类算法基于不同的假设来定义"什么是相似的点"。用错误的算法分析数据，就像用圆形的模具去切方形的饼干——结果必然不理想。

球状簇（Spherical Clusters）

簇内的点围绕一个中心均匀分布，呈现圆形或球形。

```
    ●●●●●
  ●●      ●●
 ●          ●
 ●    ●     ●   ← 中心点（质心）
 ●          ●
  ●●      ●●
    ●●●●●
```

K-Means 只能发现球状簇，因为它用质心代表簇，本质是在画"直线切割"的Voronoi图。

K-Means 的失败案例

```
月牙形数据：                    K-Means 的错误聚类：
   ●●●●●●●                         ●●●●●●●
  ●       ●                        ▲▲    ●
 ●         ●                       ▲  ●   ●
 ●         ●          →            ●   ▲  ●
  ●       ●                        ●    ▲▲
   ●●●●●●●                         ●●●●●●●

正确的簇应该是两个"月牙"        K-Means 切成上下两半
                                  （因为它只能画直线边界）
```

任意形状簇

簇的形状可以是任意的——长条形、月牙形、环形、S形等。

DBSCAN 可以发现任意形状的簇：

- 不依赖"质心"，而是看"周围点的密度"
- 只要点与点之间"够近"，就可以连成一片
- 像"顺藤摸瓜"一样，沿着高密度区域扩展

DBSCAN vs K-Means：找人类比

想象你在一个广场上找人：

方式	做法	结果
K-Means	每个人站在一个"标杆"附近，标杆在哪你就在哪	只能识别圆形区域
DBSCAN	你拉着朋友的手，朋友再拉他们朋友的手，最后连成一片	可以识别任意形状

层次结构簇

簇之间存在嵌套的层次关系——大簇中包含小簇。

生物分类学：

```
所有动物
├── 脊椎动物
│   ├── 哺乳动物
│   │   ├── 灵长类（人、猩猩）
│   │   └── 食肉类（猫、狗、熊）
│   └── 鸟类
└── 无脊椎动物
```

层次聚类专门为此设计，可以在不同层次上"切一刀"，得到不同粒度的分类。

簇形状与算法选择总结

簇的形状	典型特征	推荐算法	原因
球状簇	围绕中心均匀分布	K-Means	基于质心的分配天然适合球形
长条形簇	点沿某方向延伸	DBSCAN	密度连接
月牙形簇	弯曲的弧状结构	DBSCAN	密度聚类不受形状限制
环形簇	中心空，外围密集	DBSCAN	可以发现"空洞"结构
层次结构	大簇套小簇	层次聚类	专门为此设计

如何判断你的数据是什么形状?

1. **可视化探索**：数据是2D或3D，直接画散点图观察
2. **多种算法对比**：用不同算法跑一遍，看结果差异
3. **领域知识判断**：
    - 客户行为数据 → 可能是任意形状
    - 物理测量数据 → 可能是球状
    - 网络/社交数据 → 可能是层次结构

建议：如果不确定，优先使用 DBSCAN 或层次聚类。

第二部分：原理与算法

2.1 K-Means 聚类算法

K-Means 算法思想

核心思想："找中心、分阵营、再找中心、再分阵营.....循环直到稳定"

- **K**：表示我们想要将数据分成 K 个簇
- **Means**：表示用每个簇的"均值"（中心点）来代表这个簇

直观示例：老师把30个学生分成3个小组

1. 随机选3个学生作为"标杆"
2. 每个学生站到离自己身高最接近的"标杆"那边
3. 每个小组重新计算平均身高，选出最接近平均值的学生做新"标杆"
4. 重复2-3步，直到分组稳定

K-Means 算法步骤

1. **初始化**：随机选择 K 个点作为初始簇中心（质心）
2. **分配步骤**：每个点计算到 K 个质心的距离，分配给最近的质心
3. **更新步骤**：重新计算每个簇的质心（该簇所有点的均值）
4. **迭代**：重复步骤2和3，直到质心不再变化

目标函数：最小化 SSE（簇内平方和）

$$SSE = \sum_{j=1}^{K} \sum_{x \in C_j} \|x - \mu_j\|^2$$

SSE越小，簇内的点越集中，聚类效果越好。

K-Means 优缺点

优点：

- 简单易懂，易于实现
- 计算效率高，适合大数据集
- 对球状簇效果好

缺点：

- 需要预先指定 K 值
- 对初始值敏感
- 对异常值敏感
- 只能发现球状簇
- 不适合类别不平衡数据

K 值选择方法：肘部法则

通俗理解：找一个"性价比最高"的 K 值——再多分一类，也分不出什么新花样了。

切蛋糕类比：

- K=1：整个蛋糕一块，SSE 很大
- K=2：切成两半，SSE 大幅下降
- K=3：切成三块，SSE 又下降一些
- K=4：SSE 下降很少了

```
SSE
 ↑
 |●
 |  ●
 |    ●    ← 肘部！再往右，下降变缓
 |      ●
 |        ●—●—●
 |──────────────→ K
   1   2   3   4   5
```

K 值选择方法：轮廓系数

通俗理解：每个点都在问两个问题：

1. "我和自己簇的兄弟姐妹有多亲近？"
2. "我和隔壁簇的人有多疏远？"

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

值	含义	类比
≈ 1	内部亲近，外部疏远	"分得很对！"
≈ 0	两边都不亲不疏	"我好像在哪组都行"
≈ -1	跟外面的人更亲近	"我分错组了！"

选择原则：选择使平均轮廓系数最大的 K 值。

2.2 层次聚类

核心思想：不需要预先指定簇的数目，构建一个**树状的层次结构**（树状图）。

类比：生物分类学中的"界-门-纲-目-科-属-种"

层次聚类：班级"认亲"游戏

想象班里来了8个新同学，按"相似度"逐步抱团：

- **第一步**：各自为营（8组）
- **第二步**：找最像的两个人抱团 → 小明和小华都是学霸 → 组队
- **第三步**：组与组再合并 → 男生组合并、女生组合并
- **第四步**：继续合并 → 全班成一个大组

这就是凝聚型层次聚类：从个体开始，逐步合并最相似的群体，最终形成"家族树"。

层次聚类 vs K-Means

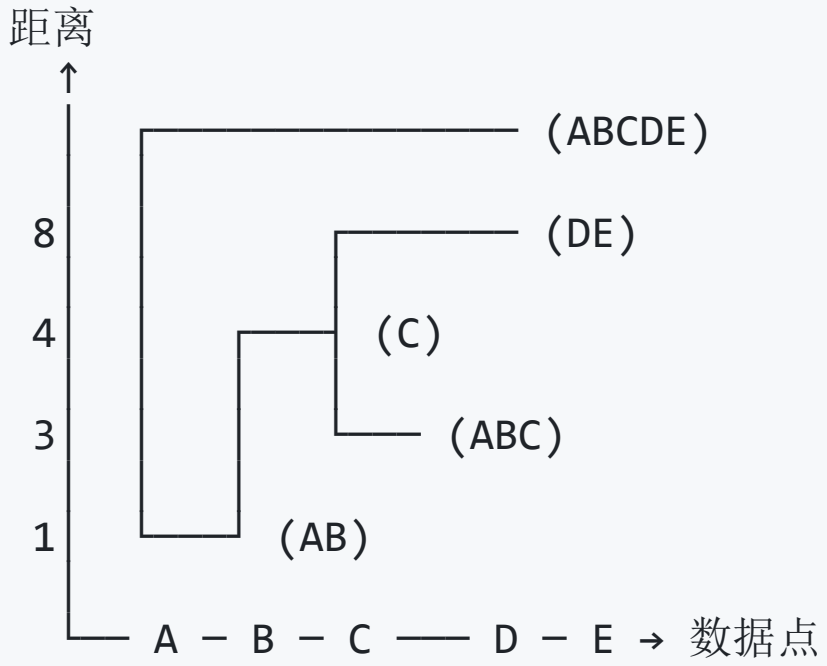
维度	K-Means	层次聚类
思维方式	"选代表，站队伍"	"找亲戚，画族谱"
决策方式	每个点选最近的中心	最相似的点/簇先合并
结果形式	扁平的K个组	嵌套的树状结构
灵活性	必须事先定K	事后随意"切一刀"
类比	选班长，班长管一片	认亲戚，从近到远

层次聚类示例：5个点

```
A     B          C                D                    E
●     ●          ●                ●                    ●
1     2          5                9                    10
```

- 第1轮：A-B距离=1（最近），合并 → (AB)，剩下4个簇
- 第2轮：D-E距离=1（最近），合并 → (DE)，剩下3个簇
- 第3轮：C离(AB)更近，合并 → (ABC)，剩下2个簇
- 第4轮：(ABC) + (DE) → 所有点成一个大簇

树状图（Dendrogram）



如何确定簇数：在某个高度画一条水平线，与树状图相交的次数就是簇的数目。

链接方法：如何计算簇间距离？

通俗理解：两个群体之间，用谁的"代表"来衡量距离？

方法	看谁	类比	适合场景
单链接	最近的两个点	看"关系最好的两个人"	链式结构
全链接	最远的两个点	看"最疏远的两个人"	紧凑簇
平均链接	所有点对平均	看"整体平均关系"	通用推荐
Ward	SSE增加最小	看"合并后内部矛盾"	球状簇

2.3 DBSCAN 密度聚类

核心思想："簇是数据空间中密度较高的区域，被密度较低的区域分隔"

与前两种算法不同：

- 可以发现**任意形状的簇**
- 能够识别**噪声点**（离群点）
- 不需要预先知道簇的数量

DBSCAN：城市人口密度类比

想象你在看一张城市的人口密度地图：

区域	人口密度	DBSCAN 判定
市中心商业区	10000人/平方公里	簇（高密度）
大学城	8000人/平方公里	簇（高密度）
郊区农田	50人/平方公里	噪声（低密度）
山区	5人/平方公里	噪声（低密度）

关键洞察：DBSCAN 不看"离市中心有多远"，而是看"你周围有多少人"。

DBSCAN：核心直觉对比

算法	核心问题	类比
K-Means	"你离哪个中心最近？"	选班长，站队
层次聚类	"谁和谁是近亲？"	画族谱
DBSCAN	"你周围人多不多？"	看人气

通俗总结：DBSCAN 就像一个"人气探测器"

- 你周围人多 → 你属于某个"人气聚集区"（簇）
- 你周围人少 → 你是"孤家寡人"（噪声点）

DBSCAN：晚会上的社交圈子

```
      ●●●●●                    ▲▲▲
    ●●      ●●●                ▲▲   ▲▲
   ●    A区     ●●            ▲    B区    ▲
   ●     (舞池)   ●           ▲    (吧台)   ▲
    ●●        ●●               ▲▲      ▲▲
      ●●●●                       ▲▲▲▲

        ●                          ◆
           ◆                          ◆
        ◆      ◆     ◆
      (稀疏区域 - 噪声点)
```

- A区（舞池）：人挤人，密度很高 → 簇
- B区（吧台）：人也很多，密度高 → 簇
- 中间稀疏区域：人很少 → 簇间分隔
- 角落几个人：周围没人 → 噪声点

DBSCAN：核心点、边界点、噪声点

DBSCAN 将数据点分为三类：

点的类型	周围人多不多?	靠近核心点吗?	类比
核心点	是（≥ MinPts）	是	人气王
边界点	否	是	边缘成员
噪声点	否	否	孤家寡人

两个关键参数：

- **ε（eps）**：邻域半径 → "你的朋友圈有多大"
- **MinPts**：最小点数 → "朋友圈里至少要有多少人"

DBSCAN 算法步骤：探索游戏

想象你在一个黑暗的大房间里探索，寻找"人群聚集区"：

1. **随机选一个起点**：看看周围有多少人？
   - 人够多（≥ MinPts）→ 发现新簇！
   - 人太少 → 暂时标记为"可能是噪声"
2. **顺藤摸瓜**：查看核心点的"朋友圈"
   - 周围人也很多 → 加入簇，继续扩展
   - 周围人不多 → 标记为边界点
3. **重复探索**：直到所有人都被访问过

DBSCAN 优缺点

优点：

- 不需要预先指定簇数
- 可以发现任意形状的簇
- 能够识别噪声点和异常值
- 对数据中的输入顺序不敏感

缺点：

- 对参数 ε 和 MinPts 敏感
- 不适合密度差异很大的数据集
- 高维数据中距离计算困难

2.4 算法对比与选择指南

特性	K-Means	层次聚类	DBSCAN
需要指定簇数	是（K值）	否	否
簇形状	球状	取决于链接方法	任意形状
对噪声敏感	是	是/否	否（可识别噪声）
适合数据规模	大	小	中
可重复性	否（初始值敏感）	是	是

如何选择算法?

选择 K-Means 的情况:

- 数据量大
- 簇大致呈球状
- 需要快速计算
- 对簇数有先验知识

选择层次聚类的情况:

- 数据量小
- 需要了解层次结构
- 需要可视化和解释性

选择 DBSCAN 的情况:

- 簇的形状可能不规则
- 数据中含有噪声/异常值

总结：关键概念回顾

概念	含义	要点
簇	一组相似对象的集合	簇内相似，簇间相异
距离度量	量化两个点相似度的方式	欧氏、曼哈顿、余弦等
标准化	消除量纲影响	Z-Score、Min-Max
质心	簇中所有点的均值	K-Means 的核心概念
核心点	密度足够高的点	DBSCAN 的基础

聚类的本质：寻找"内紧外松"的数据结构

谢谢！

聚类分析

核心要点：

1. 理解数据内在结构
2. 选择合适的距离度量
3. 根据数据结构选择算法
4. 评估聚类质量